

LABORATORI 04

Especificació de models amb SDL
Aparcament urbà

Grup: 11C
Jordi Caballeria
Marc Planas

ÍNDEX

Part A — Model bàsic	3
Q1 — Casos límit: $N_{disp} = 0$ i $N_{disp} = N$	3
Cas $N_{disp} = 0$ (aparcament ple)	3
Cas $N_{disp} = N$ (aparcament buit)	3
Disseny dels senyals de coordinació entre PGestor i PBarreraSortida	3
Traducció al diagrama de PGestor	4
Part B — Display d'estat	5
Q2 — Opcions d'implementació del display	5
Opció 1 — Emissió directa des de PGestor	5
Opció 2 — Un procés PDisplay dedicat dins de BGestor	5
Opció triada: Opció 1	5
Detecció correcta de les transicions	5
Part C — Dos tipus de plaça	6
Q3 — Opcions d'implementació de la política d'assignació	6
Opció 1 — Decisions inline al PGestor	6
Opció 2 — Procediment AssignarPlaça dins de PGestor	6
Opció triada: Opció 2 amb Procedure AssignarPlaça	6
Determinació del tipus de vehicle a PBarreraEntrada	6
Signatura del procediment AssignarPlaça	7
Tractament del tipusPlaça al VehicleSortir	7
Part D — Display amb informació de disponibilitat de carregadors	8
Transicions estrictes	8
Justificació de les tres condicions	8
Implementació al PGestor	8

Part A — Model bàsic

Q1 — Casos límit: $N_disp = 0$ i $N_disp = N$

En aquest apartat treballem amb un comptador únic N_disp que representa el nombre de places lliures, amb $N = 50$ places en total. El gestor decrementa el comptador a cada acceptació i l'incrementa a cada sortida. Cal analitzar els dos casos límit: **$N_disp = 0$** (aparcament ple) i **$N_disp = N$** (aparcament buit), tenint en compte que a la Part C el comptador es descompon en (N_est_disp , N_elec_disp) amb $total_disp = N_est_disp + N_elec_disp$. Les conclusions es generalitzen directament.

Cas $N_disp = 0$ (aparcament ple)

Quan el comptador val 0, el sistema ha d'impedir que s'accepti cap vehicle nou. En canvi, **no** s'ha de bloquejar la generació de sortides: els vehicles ja aparcats han de poder marxar normalment.

A **PGestor**: cada PeticióEntrar que arribi trobarà la condició $N_disp > 0$ com a falsa. El gestor respondrà sistemàticament amb Denegat sense modificar el comptador. El valor de N_disp no pot baixar per sota de 0 perquè la decrementació només es produeix a la branca "sí" de la decisió.

A **PBarreraEntrada**: el procés no s'atura, continua generant PeticióEntrar a intervals exponencials. La barrera no manté estat semàntic associat al resultat; rep Denegat i torna al cicle normal.

A **PBarreraSortida**: continua funcionant **normalment**. L'aparcament ple no afecta les sortides — són les entrades les que es denegen. Aquest punt és important: la condició que atura PBarreraSortida **no** és $N_disp = 0$, sinó $N_disp = N$ (cas oposat, tractat a continuació).

Cas $N_disp = N$ (aparcament buit)

Quan tots els llocs estan lliures (no hi ha cap vehicle a dins), s'ha de bloquejar la generació de sortides ja que no n'hi ha cap per marxar. Les entrades, en canvi, funcionen normalment.

A **PGestor**: la PeticióEntrar és acceptada ($N_disp > 0$ és cert). Quan N_disp passa de N a $N - 1$ (primer vehicle després d'haver estat buit), el gestor ha d'enviar el senyal de **reactivació** a PBarreraSortida.

A **PBarreraSortida**: ha d'estar a l'estat PARAT mentre l'aparcament està buit. Torna a l'estat GENERANT tan bon punt s'hi aparca un vehicle.

Disseny dels senyals de coordinació entre PGestor i PBarreraSortida

En un primer disseny vam plantejar un únic senyal AparcamentBuit interpretat amb dues semàntiques (aturar i reactivar). Aquesta dualitat és contrària a la convenció SDL: un senyal expressa un **únic** esdeveniment.

Decisió corregida: dividir-lo en dos senyals.

AturarSortides (PGestor → PBarreraSortida): emès pel gestor quan, després de processar un VehicleSortir i incrementar el comptador, detecta $N_est_disp + N_elec_disp == N$ (totes les places lliures). PBarreraSortida transita GENERANT → PARAT.

ReactivarSortides (PGestor → PBarreraSortida): emès pel gestor quan, després d'acceptar un vehicle i decrementar el comptador, detecta $N_est_disp + N_elec_disp == N - 1$ (primer vehicle que entra després d'haver estat buit). PBarreraSortida transita PARAT → GENERANT.

Avantatges del disseny:

PBarreraSortida no accedeix a cap variable del gestor (semàntica SDL pura, sense estat compartit creuat).

Cada senyal té una única semàntica, fàcil d'interpretar al diagrama i al codi.

La responsabilitat de detectar les transicions queda al PGestor, l'únic procés amb accés als comptadors.

Traducció al diagrama de PGestor

Aquest disseny es tradueix a PGestor en **dues decisions independents**, cadascuna al seu camí d'entrada:

Al camí de Acceptat (després del decrement intern del procediment AssignarPlaça), s'afegeix la decisió $N_{est_disp} + N_{elec_disp} = N - 1$?. Quan és certa, vol dir que abans del decrement total era N — el primer cotxe acaba d'entrar després d'haver estat buit. PGestor emet llavors ReactivarSortides i continua cap a la comprovació de COMPLET. Si és falsa, salta directament a la comprovació COMPLET.

Al camí del VehicleSortir (després de l'increment del comptador corresponent), s'afegeix la decisió $N_{est_disp} + N_{elec_disp} = N$? (decisió "buit"). Quan és certa, l'aparcament ha quedat completament buit i PGestor emet AturarSortides abans de tornar a INACTIU. Si és falsa, torna directament a INACTIU.

PBarreraSortida no té cap decisió: només dos inputs de senyal (AturarSortides que la passa de GENERANT a PARAT, i ReactivarSortides que la torna a GENERANT). Tota la lògica de quan emetre cada senyal queda al gestor.

Part B — Display d'estat

Q2 — Opcions d'implementació del display

Afegim el senyal `EstatAparcament` amb valors `{LLIURE, COMPLET}` cap a l'entorn, que indica si l'aparcament té alguna plaça lliure o no.

Opció 1 — Emissió directa des de PGestor

El PGestor té tota la informació de l'estat (comptadors `N_est_disp` i `N_elec_disp`). Quan detecta una transició entre `total = 0` (ple) i `total > 0` (no ple), emet `EstatAparcament(LLIURE)` o `EstatAparcament(COMPLET)` cap a l'entorn via el canal `Ch_Display` de BGestor.

Avantatges: solució mínima, sense afegir nous processos.

Inconvenients: PGestor barreja gestió i notificació.

Opció 2 — Un procés PDisplay dedicat dins de BGestor

Afegir un procés intern al bloc BGestor (anomenat PDisplay) que rep avisos interns de PGestor i els reenvia a l'entorn.

Avantatges: separació clara de responsabilitats.

Inconvenients: més processos i missatges per una funcionalitat senzilla.

Opció triada: Opció 1

Donada la senzillesa de la funcionalitat (només 2 transicions, sense filtres ni transformacions intermèdies), no es justifica el cost estructural d'un procés dedicat. Escollim **Opció 1**: PGestor emet directament `EstatAparcament` cap a l'entorn quan detecta una transició.

Detecció correcta de les transicions

El display s'emet **només en el moment de transició**, no a cada acceptació o sortida. Les dues condicions són:

COMPLET (transició $>0 \rightarrow 0$): després d'una `PeticioEntrar` Acceptada i del decrement, comprovar `N_est_disp + N_elec_disp == 0`. Si és cert, el total ha passat d' 1 a 0 (l'última plaça lliure ha estat ocupada). Emetre `EstatAparcament(COMPLET)`.

LLIURE (transició $0 \rightarrow >0$): després d'un `VehicleSortir` i de l'increment, comprovar `N_est_disp + N_elec_disp == 1`. Si és cert, el total ha passat de 0 a 1 (s'allibera la primera plaça d'un aparcament ple). Emetre `EstatAparcament(LLIURE)`.

Punt important: la comprovació té sentit només després d'un Acceptat (no Denegat). Un Denegat no modifica els comptadors i, per tant, no pot produir cap transició.

Aquestes dues transicions són **independents** dels senyals `AturarSortides` / `ReactivarSortides` de la Part A. Les transicions del display es donen al voltant del "ple" (totals 0 i 1), no del "buit" (totals $N-1$ i N).

Part C — Dos tipus de plaça

Q3 — Opcions d'implementació de la política d'assignació

S'introdueixen dos tipus de plaça: $N_{est} = 40$ (estàndard) i $N_{elec} = 10$ (amb carregador). Cada vehicle té un tipus {ELECTRIC, COMBUSTIO} i es defineix una política d'assignació amb preferència i fallback.

Opció 1 — Decisions inline al PGestor

La lògica d'assignació (intentar primer la plaça preferida segons el tipus de vehicle, després el fallback) s'expressa amb decisions encadenades directament dins del PGestor.

Avantatges: tot a la vista al diagrama del gestor; sense procediments addicionals.

Inconvenients: el flux principal del PGestor queda més pesat i barreja "acceptar o denegar" amb "quina plaça assignar"; menys reutilització.

Opció 2 — Procediment AssignarPlaça dins de PGestor

Encapsular la decisió d'assignació en un procedure cridat per PGestor des de la branca d'acceptació.

Avantatges: separació clara entre "acceptar o denegar" i "quina plaça assignar"; el flux principal queda més net; el procediment és reutilitzable.

Inconvenients: cal definir la signatura i passar paràmetres in/out.

Opció triada: Opció 2 amb Procedure AssignarPlaça

Triem **Opció 2** per modularitat i llegibilitat del flux principal.

Determinació del tipus de vehicle a PBarreraEntrada

Cal modelar explícitament la generació del tipus de vehicle (en el model bàsic aquesta tasca quedava implícita):

```
DCL double Parribada_t = 1;

double arribadaDist = 5;

double P_ELEC = 0.3; // probabilitat de vehicle elèctric

double tipus_aux;

string tipusVehicle;
```

Abans de la decisió "Tipus de vehicle" s'afegeix una tasca explícita:

```
task tipus_aux = uniform(0, 1)

decis tipus_aux < P_ELEC ?

sí → tipusVehicle = "ELECTRIC"

no → tipusVehicle = "COMBUSTIO"
```

Signatura del procediment AssignarPlaça

La signatura definitiva és:

```
procedure AssignarPlaça(in tipusVehicle,
out resultat,
out tipusPlaça)
```

El procediment retorna dos valors:

resultat: "Acceptat" o "Denegat" segons si s'ha trobat plaça.
tipusPlaça: "ELEC" o "EST" segons el tipus de plaça assignada (només significatiu si resultat = "Acceptat"). Imprescindible perquè PGestor pugui mantenir els comptadors separats i emetre EstatCarregadors (Part D).

Quatre branques principals (més rebuig):

Vehicle ELECTRIC i $N_{elec_disp} > 0 \rightarrow$ decrementa N_{elec_disp} ; tipusPlaça = "ELEC";
resultat = "Acceptat".
Vehicle ELECTRIC, $N_{elec_disp} = 0$ i $N_{est_disp} > 0 \rightarrow$ decrementa N_{est_disp} ; tipusPlaça = "EST"; resultat = "Acceptat" (fallback).

Vehicle COMBUSTIO i $N_{est_disp} > 0 \rightarrow$ decrementa N_{est_disp} ; tipusPlaça = "EST";
resultat = "Acceptat".

Vehicle COMBUSTIO, $N_{est_disp} = 0$ i $N_{elec_disp} > 0 \rightarrow$ decrementa N_{elec_disp} ;
tipusPlaça = "ELEC"; resultat = "Acceptat" (fallback).

En cas contrari $\rightarrow$ resultat = "Denegat" i tipusPlaça queda indefinit.

Tractament del tipusPlaça al VehicleSortir

PBarreraSortida no té cap manera fiable de saber el tipus de plaça d'un vehicle que marxa: la barrera no manté inventari ni accedeix a les variables del gestor. Per tant, el senyal VehicleSortir es manté **sense paràmetres**.

PGestor mostreja internament tipusPlaça quan rep VehicleSortir, ponderant pel nombre real de places ocupades de cada tipus:

```
task occupied_elec = N_elec - N_elec_disp
task occupied_est = N_est - N_est_disp
task total_oc = occupied_elec + occupied_est // > 0 garantit
task tipus_aux2 = uniform(0, 1)
decis tipus_aux2 < (occupied_elec / total_oc) ?
sí  $\rightarrow$  tipusPlaça = "ELEC";  $N_{elec\_disp} = N_{elec\_disp} + 1$ 
no  $\rightarrow$  tipusPlaça = "EST";  $N_{est\_disp} = N_{est\_disp} + 1$ 
```

El repartiment de tipus de plaça segueix la proporció real d'ocupació, sense necessitat de comunicar tipusPlaça pel senyal des de la barrera. La condició $total_oc > 0$ és sempre certa: si fos 0, PBarreraSortida estaria a PARAT i no emetria VehicleSortir.

Part D — Display amb informació de disponibilitat de carregadors

S'afegeix el senyal `EstatCarregadors` amb valors {DISPONIBLE, EXHAURIT} cap a l'entorn. Es modela amb les mateixes consideracions de **transició estricta** aplicades a `EstatAparcament` (Part B).

Transicions estrictes

EstatCarregadors(EXHAURIT) — transició `DISPONIBLE` → `EXHAURIT`.

Emès si i només si es compleixen totes tres condicions:

1. Acabem de processar un `Acceptat de PeticióEntrar` (camí del decrement).
2. `tipusPlaça == "ELEC"` (el procediment ha assignat efectivament una plaça elèctrica).
3. `N_elec_disp == 0` **després** del decrement.

EstatCarregadors(DISPONIBLE) — transició `EXHAURIT` → `DISPONIBLE`.

Emès si i només si es compleixen totes tres condicions:

1. Acabem de processar un `VehicleSortir` (camí de l'increment).
2. `tipusPlaça == "ELEC"` (la plaça alliberada és elèctrica, segons el mostreig intern).
3. `N_elec_disp == 1` **després** de l'increment.

Justificació de les tres condicions

Sense la **primera** (camí correcte): `EXHAURIT` només pot disparar-se en decrement; `DISPONIBLE` només en increment.

Sense la **segona** (`tipusPlaça == ELEC`): el mateix `Acceptat` podria disparar un fals `EXHAURIT`. Per exemple, si una `COMBUSTIÓ` pren una plaça estàndard mentre `N_elec_disp` ja era 0, **no** és transició — l'`EXHAURIT` s'havia anunciat anteriorment.

Sense la **tercera** (igualtat exacta amb 0 o 1): el senyal es repetiria a cada acceptació/sortida; no seria informació de transició.

Implementació al PGestor

Path Acceptat (després del decrement i de la comprovació COMPLET):

```
decis tipusPlaça = "ELEC" AND N_elec_disp = 0 ?
sí → EstatCarregadors(EXHAURIT) TO env
no → continuar
```

Path VehicleSortir (després de l'increment, dins la branca `tipusPlaça = ELEC`):

```
(la branca "tipusPlaça = ELEC" del mostreig intern ja garanteix la condició 2)
decis N_elec_disp = 1 ?
sí → EstatCarregadors(DISPONIBLE) TO env
no → continuar
```

Aquesta organització manté els displays de `EstatAparcament` i `EstatCarregadors` alineats: ambdós s'emeten estrictament a les transicions corresponents, sempre després d'una acceptació o d'una sortida — mai a la branca `Denegat`.